

Writing an OS in RUST

<https://os.phil-opp.com/>

Semaine 1 : Fondations et Intégrité du Noyau

Jour	Thème	Activité (3h)	Focus Sécurité
1	Environnement	Installer Rust (nightly), cibles bare-metal et cargo-xbuild.	Comprendre la surface d'attaque d'une chaîne de compilation.
2	No Standard	Créer un binaire <code>#![no_std]</code> qui ne dépend d'aucun OS.	Pourquoi l'absence de bibliothèque standard réduit les vulnérabilités.
3	Panic & Crash	Implémenter un gestionnaire de panique (<code>panic_handler</code>).	Gérer les erreurs fatales sans laisser le système dans un état instable.
4	VGA Buffer I	Accéder à la mémoire vidéo (0xb8000) et écrire un caractère.	Manipulation directe du matériel et risques de corruption mémoire.
5	VGA Buffer II	Créer une interface sécurisée (Safe Wrapper) et une macro <code>println!</code> .	Comment Rust empêche les accès concurrents non sécurisés au matériel.
6	Tests Kernels	Mettre en place des tests unitaires automatisés avec QEMU.	L'importance des tests pour prévenir les failles logiques.
7	Exceptions CPU	Créer la table d'interruption (IDT) pour les erreurs CPU.	Analyser comment un CPU signale une violation de segment.

Semaine 2 : Mémoire, Isolation et Multitâche

Jour	Thème	Activité (3h)	Focus Sécurité
8	Double Faults	Gérer les fautes doubles avec une pile de secours (IST).	Prévenir le débordement de pile (Stack Overflow) qui fait crash l'OS.
9	Pagination I	Théorie des 4 niveaux de tables de pages x86_64.	Comprendre l'isolation matérielle entre processus.
10	Pagination II	Implémenter le parcours des tables de pages dans le noyau.	Le bit NX (No-Execute) : empêcher l'exécution de code dans les données.
11	Allocation I	Créer un allocateur de frames (pages physiques).	Prévenir la fragmentation et l'épuisement des ressources.
12	Allocation II	Mapper le "Heap" (tas) pour permettre	Différence entre allocation statique et

Jour	Thème	Activité (3h)	Focus Sécurité
		l'allocation dynamique.	dynamique en sécurité.
13	Heap Allocator	Utiliser des bibliothèques (Linked List, Fixed Size Block).	Analyser les risques de "Use-after-free" (et comment Rust les évite).
14	Async/Await	Créer un exécuteur de tâches simple pour le clavier.	Multitâche coopératif vs préemptif : quel impact sur la sécurité ?
15	Synthèse	Démonstration finale : Un OS stable qui gère clavier, écran et mémoire.	Revue de code : où sont les blocs unsafe restants et pourquoi ?